

CS3.301 Operating Systems and Networks

Mini Project 1 — MLFQ Scheduler for xv6

1. Introduction

The xv6 kernel originally uses a round-robin (RR) scheduler. For this part of Mini Project 1, an additional Multi-Level Feedback Queue (MLFQ) scheduler was implemented and integrated into xv6. The scheduler is selected at compile time using the `SCHEDULER` macro, while leaving the original RR path unchanged when no scheduler is specified.

The implementation follows the required four-level MLFQ design. Queue 0 is the highest-priority queue and queue 3 is the lowest-priority queue, with time slices of 1, 4, 8, and 16 timer ticks respectively. New processes enter queue 0, CPU-bound processes are progressively demoted when their time slices expire, queue 3 is scheduled in round-robin fashion, and all processes receive a priority boost to queue 0 every 48 ticks.

2. Implementation Summary

2.1 Makefile and SCHEDULER Selection

The xv6 Makefile was modified so that the scheduler can be selected at compile time through the `SCHEDULER` macro.

The default build continues to use the original xv6 round-robin scheduler:

```
make clean
make qemu
```

The MLFQ scheduler is selected using:

```
make clean
make qemu SCHEDULER=MLFQ
```

The scheduler-specific code is conditionally compiled using the `SCHEDULER_MLFQ` configuration. Therefore, leaving `SCHEDULER` unset preserves the original RR scheduling path.

2.2 struct proc Changes

Additional bookkeeping fields were added to each process to maintain its MLFQ state. The important fields are:

- `queue` — the process's current MLFQ queue.
- `slice_ticks` — number of ticks consumed in the current queue time slice.
- `ticks_since_boost` — ticks elapsed since the last priority boost.
- `enqueue_seq` — sequence number used to preserve queue insertion order.
- `run_ticks` — cumulative CPU ticks received by the process, useful for measuring service time.

These fields allow the scheduler to track both the current priority of a process and the information required to implement time-slice exhaustion, queue ordering, and periodic boosting.

2.3 allocproc() and Process Creation

When a new process is allocated, its MLFQ bookkeeping is initialized. A newly created process starts in queue 0 with zero ticks consumed in its current slice and zero ticks since the last boost.

When the process becomes runnable, it is inserted at the tail of queue 0. The same placement rule is applied to newly created child processes.

An enqueue sequence number is assigned whenever a process is inserted into the runnable queue. This provides deterministic ordering among processes at the same priority.

2.4 Queue Selection and Strict Priority Scheduling

The MLFQ scheduler scans the queues from queue 0 through queue 3 and selects a runnable process from the highest-priority non-empty queue.

Within a queue, the runnable process with the earliest enqueue sequence number is selected. This preserves FIFO ordering within queues 0–2 and gives queue 3 the required round-robin behaviour as processes repeatedly reach the tail of that queue.

The scheduler therefore implements strict priority: a runnable process in a higher-priority queue is always selected before processes waiting in lower-priority queues.

2.5 Time-Slice Handling and Preemption

The four MLFQ time slices are:

Queue	Priority	Time Slice
Q0	Highest	1 tick
Q1		4 ticks
Q2		8 ticks
Q3	Lowest	16 ticks

At every timer tick, the running process's `slice_ticks` counter is updated. Once the process consumes its complete time slice, it is preempted.

For queues 0, 1, and 2, time-slice exhaustion moves the process to the tail of the next lower queue:

Q0 → Q1 → Q2 → Q3

A process already in Q3 remains in Q3 and is placed at its tail, giving the lowest-priority queue round-robin scheduling.

The implementation also checks for the presence of runnable processes in higher-priority queues. Thus, when a higher-priority process becomes runnable, the currently running lower-priority process can be preempted at the next timer-tick boundary.

2.6 Voluntary Yield

A process that voluntarily yields before exhausting its current time slice does not get demoted.

Instead, it remains at the same priority and is reinserted at the tail of its current queue when it becomes runnable again.

Therefore:

- time-slice exhaustion causes demotion;
- voluntary yielding preserves the current queue.

This distinction is important for the feedback behaviour of the scheduler because CPU-bound processes are gradually moved downward, while processes that frequently give up the CPU before their slice expires retain their priority.

2.7 Priority Boosting

To prevent starvation of processes that have reached lower-priority queues, a global priority boost is performed every 48 timer ticks.

During a boost, every process in the system is moved to queue 0. Its current slice counter and boost counter are reset, and it receives a new enqueue sequence number.

The implementation uses a pending-boost mechanism so that the periodic timer interrupt records the boost event and the scheduler performs the corresponding queue updates safely.

In the experimental run, priority boosts were observed at ticks 49 and 101 during the scheduler test workload. These events can be seen clearly in Figure 1.

2.8 procdump() Changes

procdump() was extended to expose MLFQ-specific scheduler information. For each process, the output includes:

```
PID  STATE  NAME  Q  SLICE  BOOST  RUN
```

where:

- Q is the current queue,
- SLICE is the number of ticks consumed in the current slice,
- BOOST is the number of ticks since the last priority boost,
- RUN is the accumulated CPU time.

This provides a convenient way to inspect the scheduler state using the xv6 console's Ctrl-P facility and was used during development to verify queue movement and priority boosting.

3. MLFQ Analysis

3.1 Experimental Workload

A custom scheduler test user program was written because the assignment's MLFQ test is not part of the default xv6 distribution.

The test creates four CPU-bound child processes with different amounts of computational work. Each process records its start and completion time, while the kernel was temporarily instrumented to record the process ID, current queue, and slice state at each timer tick.

The experiment was run with a single CPU so that the observed queue transitions directly represent the scheduler's decisions.

3.2 Experimental MLFQ Execution Trace

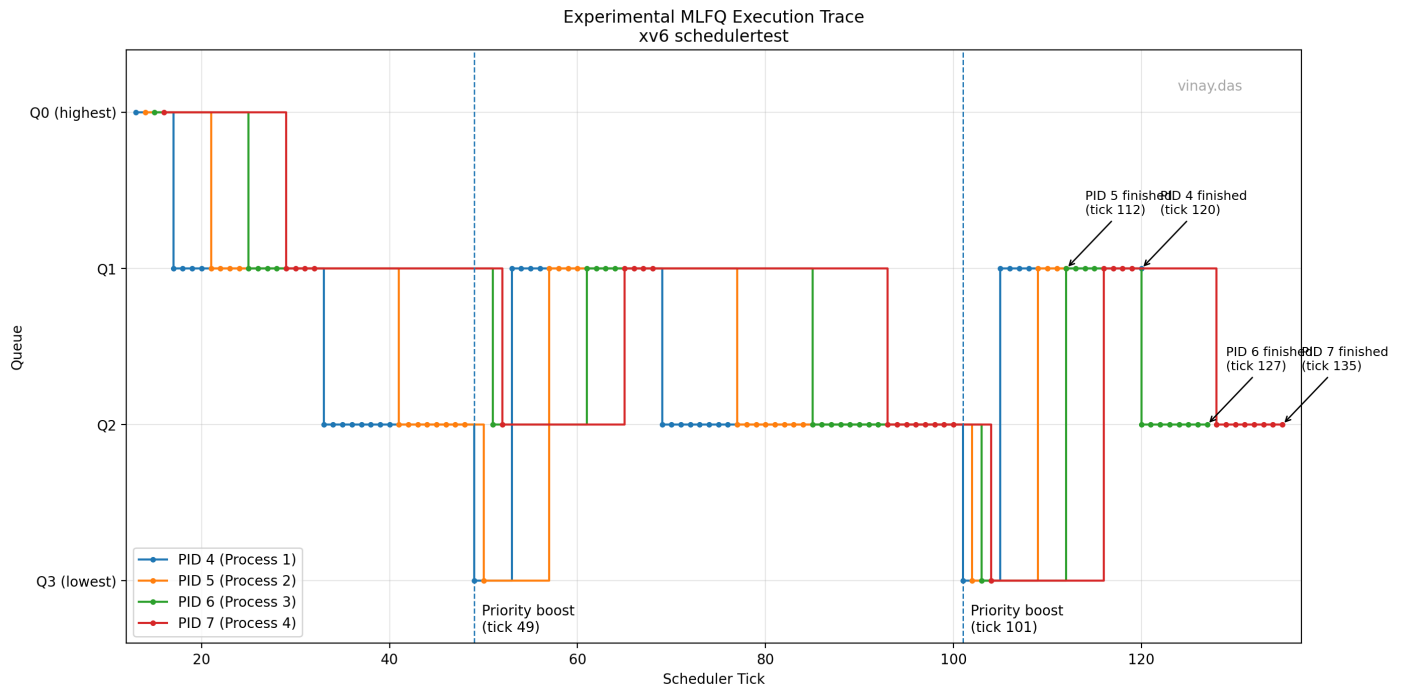


Figure 1: Experimental MLFQ execution trace obtained from the xv6 scheduler test workload. The X-axis measures elapsed time from the beginning of the workload, while the Y-axis shows the current MLFQ queue. Each process is represented by a separate colour. Dashed vertical lines indicate the observed 48-tick priority boosts.

The experiment shows all four processes initially entering the highest-priority queue Q0. As the CPU-bound processes exhaust their allotted slices, they move progressively through Q1, Q2, and Q3, demonstrating the feedback mechanism of the scheduler. The priority boosts observed at ticks 49 and 101 return the active processes to Q0, after which they again move down the queues as they consume CPU time. The longer-running processes spend more time in the lower queues, while shorter jobs complete earlier. This demonstrates both the intended demotion behaviour and the anti-starvation effect of periodic priority boosting.

4. Cross-Scheduler Comparison

The same fixed set of four CPU-bound processes was used to compare FIFO, Round Robin, and MLFQ. The measured metrics are:

- **Response time:** time from process arrival until its first CPU allocation.
- **Turnaround time:** time from process arrival until completion.
- **Waiting time:** turnaround time minus the CPU service time.

The workload used increasing CPU demands for the four processes. The measured results were averaged over the four processes.

4.1 Results

Scheduling Policy	Average Response Time	Average Turnaround Time	Average Waiting Time
FIFO	6.50	14.75	6.50
Round Robin	1.50	20.50	12.50
MLFQ	1.50	17.25	9.25

Scheduler Performance Comparison

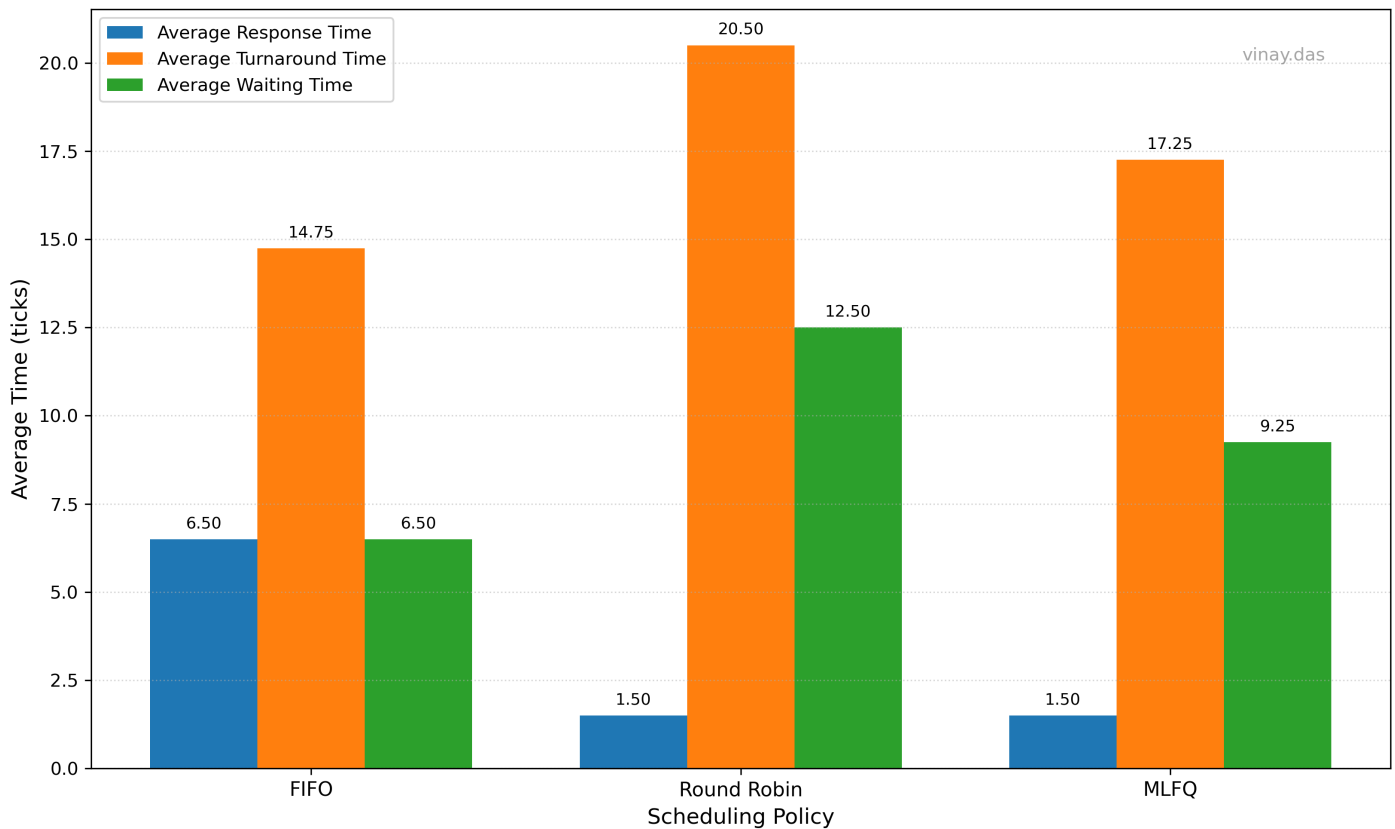


Figure 2: Comparison of average response, turnaround, and waiting times for FIFO, Round Robin, and MLFQ on the same workload.

FIFO has the highest average response time because later processes must wait for earlier processes to complete before receiving CPU time. However, because the processes arrive together in the chosen order, FIFO performs well on average turnaround and waiting time for this particular workload. Round Robin provides much better response time because CPU time is shared among the processes, but its repeated rotation causes higher average waiting and turnaround times for this workload. MLFQ achieves the same average response time as Round Robin while reducing average turnaround time from 20.50 to 17.25 ticks. It also reduces average waiting time from 12.50 to 9.25 ticks compared with Round Robin. The improvement comes from allowing short or recently boosted processes to receive higher-priority service while progressively moving CPU-bound processes toward lower-priority queues. Therefore, on this workload, MLFQ provides a better balance between responsiveness and completion time than Round Robin, although FIFO happens to achieve the lowest turnaround and waiting times because of the particular arrival order and burst pattern used in the experiment.

5. Conclusion

The xv6 scheduler was extended with a compile-time selectable MLFQ policy while retaining the original round-robin behaviour when no scheduler is specified. The implementation maintains four priority queues with progressively larger time slices, performs strict-priority scheduling, demotes CPU-bound processes after slice exhaustion, preserves priority on voluntary yields, and periodically boosts all processes to Q0 to prevent starvation. The experimental trace confirms that processes move through the expected queues and return to Q0 at the periodic boost points. The cross-scheduler experiment also shows that MLFQ provides significantly better turnaround and waiting times than Round Robin on the chosen workload while maintaining the same average response time. The results illustrate the main trade-off of MLFQ: it attempts to combine the responsiveness of preemptive scheduling with preferential treatment for processes that do not continuously consume CPU time.

6. Experimental Data Summary

MLFQ trace

- Workload start: tick 13
- Priority boost: tick 49
- Priority boost: tick 101
- Process 2 (PID 5) completed: tick 112
- Process 1 (PID 4) completed: tick 120
- Process 3 (PID 6) completed: tick 127
- Process 4 (PID 7) completed: tick 135

Scheduler comparison

Scheduler	Response	Turnaround	Waiting
FIFO	6.50	14.75	6.50
RR	1.50	20.50	12.50
MLFQ	1.50	17.25	9.25